THE ANSWERS

A: Analysis, B: Bookwork, C: Computed Example, D: Design

1. a) Figure 1.1 shows the timing of CPU bursts (or *jobs*) from four processes *A* - *D* running under an operating system. You may assume that no other CPU bursts happen between times of 0 and 11 milliseconds, and that no other processes exist in the system. Each job is labelled with the name of its process.

   i) For each scheduling method: first come first served, shortest remaining job first, and priority scheduling with pre-emption, draw a diagram showing the scheduling of these four processes between times 0 and 11 milliseconds. Process priorities, where needed, are as shown in Figure 1.1. State clearly which scheduling method applies to which diagram, and indicate clearly the times at which the running process changes.

   [3]

   ii) Processes *A* - *D* are created and admitted to the system to run at the arrival time of the job as shown in Figure 1.1. Processes *A* and *B* terminate at the end of their CPU burst, processes *C* and *D* perform an I/O system call at the end of their CPU burst. State, for each scheduling method used in part (i), which processes are in the operating system ready list, and which in the operating system waiting list, at a time of 6.5 milliseconds.

   [3]

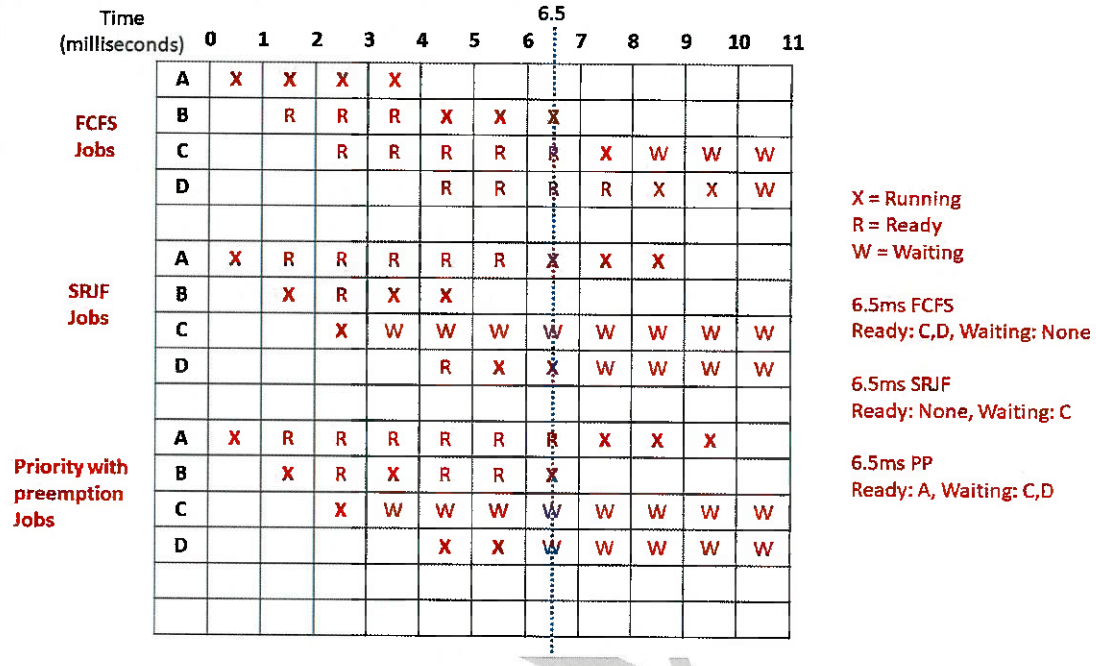
Job	Priority	Arrival time (milliseconds)	Duration (milliseconds)
A	4	0	4
B	3	1	3
C	2	2	1
D	1	4	2

Figure 1.1: Four jobs. *D* has the highest priority.

Answers

A & B.
1 mark each for correct diagrams
1 mark each for correct ready and waiting lists for the three scheduling methods.

Diagrams and lists are shown below. Note that marks for diagrams are available if the process run times (marked with X) are correctly indicated, ready/waiting indications as here are not required, although useful, because they allow ready and waiting lists to be computed.



b) A semaphore is implemented with three functions: `init(s,count)`, `wait(s)`, and `signal(s)`.

i) State how the semaphore is initialised if it is used to implement mutually exclusive critical regions in three processes *A*, *B* and *C*.

[1]

ii) State how the semaphore is initialised if it is used to ensure that process *A* critical region happens before process *B* critical region.

[1]

iii) If the semaphore count is *n* what is known about the number of semaphore waiters?

[1]

iv) Figure 1.2 shows one implementation of the semaphore `signal` and `wait` functions. For each function, state whether atomic execution of any part of the code is required, and if so identify the smallest such part.

[2]

v) Compared with semaphores, give one advantage, and one disadvantage, of Peterson's algorithm in solving mutual exclusion problems.

[1]

vi) Processes *A* and *B* execute with other processes under an operating system, and use Peterson's algorithm to ensure mutual exclusive access to a critical region. Figure 1.3 shows the code of Peterson's algorithm and a dimensioned trace of *A* and *B* each executing one critical region. The waveforms and times are as in Figure 1.3. Why is it not possible for the critical region of B to be executed before that of A? What is the maximum and minimum possible length of the critical region of process *B*?

[2]

B.
(i), (ii),(iii)
Mutual exclusion: `init(s, 1)`
A before B: `init(s,0)`
If $n \geq 0$, no waiters. If $n < 0$, $-n$ waiters.

(iv)

```
wait(s)
{
    /* start atomic */
    s.count = s.count - 1
    if s.count < 0 then {
    /* end atomic */
       add current process to waiters list
       suspend current process
    } else {
    /* end atomic */
    }
}

signal(s)
{
    /* start atomic */
    s.count = s.count + 1
    if s.count <= 0 then {
    /* end atomic */
       remove one process from waiters list
       unsuspend removed process
    } else {
    /* end atomic */
    }
}
```

(v)

Advantage: does not require hardware support
Disadvantages (one required):
(1) Complex (and impossible as taught) for more than two processes
(2) Incorporates busy/wait loop which is inefficient

(vi)

Interested_A is true before the Process *B* while loop starts. Turn is set to A by Process *B*. Therefore the Process *B* while loop terminates after Interested_A becomes false, at which point *A* critical region will have completed. Process B critical region cannot execute until process B while loop has completed.

Max *B* critical region time: 8 - 5 = 3
Min *B* critical region time: 0 (B could be suspended for any amount of the available time).

Answers

```
wait(s)
{
    s.count = s.count - 1
    if s.count < 0 then {
       add current process to waiters list
       suspend current process
       }
}

signal(s)
{
    s.count = s.count + 1
    if s.count <= 0 then {
       remove one process from waiters list
       unsuspend removed process
       }
}
```

Figure 1.2: Semaphore operations

Process A

```
Interested_A = TRUE;
Turn = 'B';
While (Interested_B=TRUE AND Turn='B')
            do_nothing;
critical region;
Interested_A=FALSE;
```

Process B

```
Interested_B=TRUE;
Turn = 'A';
While (Interested_A=TRUE  AND Turn = 'A')
            do_nothing;
critical region;
Interested_B=FALSE;
```

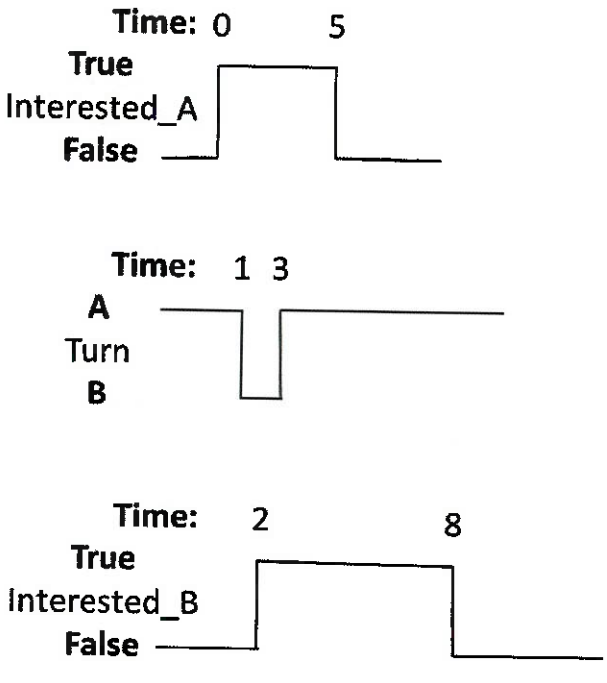


Figure 1.3: Peterson's Algorithm.

©Imperial College London

c) This question relates to the condition of deadlock in an operating system

i) Define a resource allocation graph and state how this can be used to determine the presence or absence of deadlock.

[2]

ii) Figure 1.4 shows the use of 15 identical system resources by processes A, B, C, D at two different times Q and R, and the known maximum resource usage of each process. Using the Banker's algorithm, for Q and R, either prove the state safe or show that it cannot be safe.

[2]

iii) State a general condition on the order of mutually exclusive resource usage by processes that will ensure deadlock can never happen. Figure 1.5 shows code in which four semaphores S_a, S_b, S_c, S_d provide mutually exclusive access to resources a, b, c, d used by three processes $P1, P2, P3$. Enumerate *all* the possible correct sequences of semaphore operations used by $P3$ before its critical region $CR3$ that can ensure safe use of resources without deadlock.

[2]

Question 1. has a total of 20 marks.

Process	Maximum usage	Usage at Q	Usage at R
A	13	6	7
B	7	3	3
C	1	1	1
D	6	5	4

Figure 1.4: Process resource use.

$P1$	$P2$	$P3$
wait(S_a)	wait(S_c)	?
wait(S_d)	wait(S_b)	
Critical region CR1	*Critical region CR2*	*Critical region CR3*
signal(S_a)	signal(S_b)	signal(S_a)
signal(S_d)	signal(S_c)	signal(S_b)
		signal(S_c)

Figure 1.5: Process semaphore use.

(i) A
Resource graph: A directed graph with nodes for resources and processes, and arrows from process to resource it is waiting to require, and resource to process that currently holds it in a mutex.

A cycle in the resource graph indicates <=> deadlock
(ii) C
safe state sequences
$Q: C \rightarrow D \rightarrow B \rightarrow A \Rightarrow Q$ is a safe state
$R: C \rightarrow$ (no next state possible) $\Rightarrow R$ is not a safe state

(iii)B then A
In the general case all resources must be given a definite total global order. Then, when a process holds resources and requests a new resource x, all resources held must be strictly before x in this order.
In this case:
(1) P3 must wait on Sa, Sb, Sc.
(2) Of the 6 possible orders of the wait statements we can find allowed orders as follows
Determine global order of a,b,c,d. We have $a < d$, $c < b$, from P1 and P2 wait orders. Therefore possible orders are:
$a < d < c < b$
$a < c < d < b$
$a < c < b < d$
$c < a < d < b$
$c < b < d < a$

Sd is not used, therefore we have three `wait()` calls in allowed orders:
$a < c < b$
$c < a < b$
$c < b < a$

Example: $c < a < b$ corresponds to:

```
wait(Sc)
wait(Sa)
wait(Sb
```

Answers